

Data Structures and Algorithms -3 (25CS2103E)

Delivery Route Optimizer

Team Members

R.RadhaKrishna 2520090063

R.Jagadeesh 2520090030

Under the Esteemed Guidance of

**Name of the DSA-3 faculty name with
designation and department**

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope



Abstract

- Java-based system that finds the shortest and most cost-effective delivery routes between multiple locations
- Applies core DSA concepts — graph representation and Dijkstra's Algorithm — to compute the optimal path
- Takes delivery locations, distances and destination details as input from the user
- Uses graphs, priority queues and efficient searching techniques to solve real-world logistics problems
- Provides a simple, user-friendly interface for entering routes and viewing optimized results
- Reduces unnecessary travel, saving time, fuel and operational cost

Introduction

- Efficient delivery routing is critical for modern logistics, e-commerce and transportation services
- Manual route planning is slow, error-prone and rarely gives the most cost-effective path
- The Delivery Route Optimizer automates route selection using Data Structures and Algorithms
- The delivery network is modeled as a weighted graph — locations are nodes, distances are edges
- Goal: determine the shortest path between a source and multiple delivery destinations
- Built in Java, combining a strong algorithmic foundation with a practical, usable application



Literature Survey

- Dijkstra's Algorithm (1959): a greedy approach that finds the shortest path from a single source to all other nodes
- Bellman-Ford Algorithm: handles graphs with negative edge weights, but is slower than Dijkstra's Algorithm
- A* Search Algorithm: a heuristic-based approach, commonly used in GPS and navigation systems
- Commercial systems such as Google Maps and delivery apps rely on similar graph-based shortest-path techniques
- Using a Priority Queue (Min-Heap) to implement Dijkstra's Algorithm improves its running time considerably
- Existing literature supports graphs and shortest-path algorithms as an effective foundation for route optimization



Problem Statement and Objectives

Problem Statement

- Manual and unplanned delivery routing leads to wasted travel distance, higher fuel usage and increased delivery time and cost

Objectives

- Design a graph-based model to represent delivery locations and routes
- Implement Dijkstra's Algorithm to compute the shortest path efficiently
- Build a simple interface to input locations, distances and destinations
- Minimize total travel distance and overall delivery time
- Display a clear, optimized delivery route as the final output



Flow Chart

Input Locations, Distances & Destinations



Build Weighted Graph (Nodes & Edges)



Apply Dijkstra's Algorithm using a Priority Queue



Calculate the Shortest / Optimal Path



Display Optimized Route & Total Distance



Results

- Successfully computes the shortest path between a source location and multiple delivery destinations
- Correctly handles multiple locations connected by routes of varying distances
- Displays the optimized delivery route along with the total distance covered
- Route generated by the system shows reduced travel distance compared to manual/unordered selection
- The interface accepts dynamic input of new locations, distances and destinations
- Output of Dijkstra's Algorithm was verified and matched manual shortest-path calculations

Hardware and Software requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 500 MB free space
- Standard keyboard, mouse and monitor

Software Requirements

- Operating System: Windows / Linux / macOS
- Programming Language: Java (JDK 8 or above)
- IDE: Eclipse / IntelliJ IDEA / NetBeans
- Java Collections Framework (PriorityQueue, HashMap, ArrayList)



Conclusion

- The Delivery Route Optimizer successfully applies Data Structures and Algorithms to a real-world logistics problem
- Demonstrates the practical use of graphs, priority queues and shortest-path algorithms
- Helps reduce travel distance, delivery time, fuel consumption and operational cost
- Strengthens understanding of algorithm design, graph theory and optimization techniques
- Delivers a simple, functional and educational route-management solution

Future Scope

- Integrate real-time traffic data for dynamic, up-to-date route updates
- Extend the system to multi-vehicle, multi-destination routing (Vehicle Routing Problem)
- Add GPS and map API integration for real-world coordinates and visualization
- Implement the A* algorithm for faster, heuristic-based pathfinding
- Develop a mobile or web application for wider accessibility
- Incorporate machine learning to predict demand and traffic patterns

Thank you